

Latent-Space Simulation Meets a Real Adversary: A World-Model Approach to Kaggle ConnectX

A reinforcement-learning world model, real adversarial search, and an exact endgame solver applied to Kaggle's ConnectX competition.

Alexandros Titonis | Competition: <https://kaggle.com/competitions/connectx>

1. What this is, and what it isn't

The core idea: instead of generating a move token-by-token or via a hand-written heuristic, a small learned model (encoder + latent dynamics + value head + a diagnostic decoder) rolls forward candidate moves in latent space, and a search procedure picks the move whose imagined future looks best -- MuZero-style planning [1].

This is one instance of a GENERAL latent-space-simulation architecture, not a Connect-4-specific system: the same encoder/dynamics/value/decoder + search pattern used here has also been applied to algebraic equation solving, constraint-satisfaction-style logic puzzles, and route-planning problems, with zero domain-specific changes to the core training/search code -- swapping domains means implementing one small interface (see `environment.py`), not rewriting the architecture. ConnectX is the instance documented in this paper because it is the first genuinely ADVERSARIAL two-player domain the architecture was applied to: a single-agent puzzle solver only ever has to be right, while a Connect-4 agent has to be right against an opponent actively trying to make it wrong. That one change breaks assumptions the rest of the pipeline had never been asked to question, and is the actual subject of this write-up.

This is NOT a hand-written Connect-4 bot. The board simulator and rules are exact (no need to make a neural network learn something 40 lines of Python computes for free), but every DECISION -- what a position is worth, which past games are relevant, when the value head should update online -- comes from a learned model trained without a Connect-4-specific oracle.

1.1 How this compares to typical RL game-playing systems

Systems in the AlphaZero/MuZero lineage [1][2] are normally trained at large scale: thousands of self-play games generated in parallel across many machines (often TPU or GPU clusters), running for days. This project's result was trained on a single consumer laptop -- see Section 1.2 for the exact specs -- with a full training-pipeline run (stage 1 + stage 2 + self-play) confirmed at roughly one hour of wall-clock time, and on the order of tens of thousands of self-play games, not millions. This is presented honestly as evidence that the ARCHITECTURE (real search over exactly-known rules, substituting for an expensive learned-world-model rollout) is a genuinely efficient way to reach competent play in a domain with known rules -- not a claim that this matches AlphaZero-level strength; Section 6 states plainly where this still falls short of a fully-solved or superhuman opponent.

1.2 Training hardware

Every result in this paper was produced on a single machine: an RTX 4060 laptop GPU, a Ryzen AI 9 HX 370 CPU, and 32GB RAM. No cluster, no cloud training run, no multi-GPU parallelism -- everything in this repository, including a full from-scratch training run, was reproduced on that one laptop.

2. Architecture

2.1 The easy path: collapse the adversary into an ordinary MDP

The cheapest way to test whether this works at all: bake a fixed, non-learning opponent policy directly into the environment's `step()` function. From the agent's perspective this is then indistinguishable from a single-agent puzzle -- it picks an action, the environment returns the resulting state (already reflecting the opponent's reply). Zero changes needed to the rest of the pipeline. At the real 7x6 board scale there is no exact oracle (the game tree is too large for brute-force search), so the value head is trained via on-policy Monte Carlo returns instead of regression against oracle labels.

2.2 Why the dynamics model can't imagine "after my move, before theirs"

`step()` bundles the agent's move and the fixed opponent's reply into ONE transition, so the trained dynamics model only ever saw full round-trips as single training examples -- it structurally cannot represent "the board immediately after my move, before their reply" as a state, because it never saw that state shape. Asking it to imagine that intermediate state would be extrapolating outside its training distribution.

2.3 The fix: real search over the real board, learned value only at the leaf

Since Connect-4's rules are exactly known, there's no need to make the network imagine anything mid-ply. The deployed search does the adversarial ply in REAL board space: enumerate the agent's real legal moves; for each, enumerate the opponent's real legal replies and assume they pick whichever hurts the agent most (a genuine minimax, not a single guess). The learned value head is used only as the LEAF evaluator, on a real, never-imagined state. This is structurally closer to AlphaZero's design (real search + a learned value net, viable because the rules are exactly known) [2] than MuZero's (search over a LEARNED model, needed specifically when true dynamics are not available) [1].

Result of this one architectural change: 95-100% win rate vs. random play, 100% vs. the fixed weak training heuristic, up to 66.7% vs. a stronger 1-ply-deeper heuristic never seen during training -- beating every latent-search configuration tried on every metric simultaneously (the best latent-search result had been 75% / 50% / 11.7%).

2.4 The exact endgame solver

Real search over real board states opens a door latent-space search never could: once a position narrows down to a handful of legal columns -- which happens naturally as the board fills -- the remaining game tree is small regardless of how many plies are left, and can be SOLVED EXACTLY, no learned value head, no guessing. The solver is a memoized alpha-beta minimax with center-out move ordering, triggered whenever 5 or fewer columns remain legal (calibrated empirically against real Kaggle replay data: 5 columns / 24 empty cells solves in under half a second; 6 columns did not finish inside a 5-second cap with this plain-Python implementation). It is bounded by a hard wall-clock deadline as an independent safety net -- a position outside the calibrated safe zone falls through to the round-based search rather than risking the real move-time budget.

The insight that made this cheap: branching factor is controlled by how many columns are LEGAL, not by how many cells are EMPTY. A position can have twenty or more empty cells and still be trivial to solve exactly if only a few columns remain open -- exactly the shape a real Connect-4 endgame takes

once most of the board has filled.

2.4a The bitboard solver -- and how a 92.5%-agreeing implementation was caught being wrong

The solver above is plain Python with a memo keyed on the full board tuple, and its own docstring gates callers at 5 or fewer legal columns because that is its practical reach. Two things were blocked behind that gate: a strong enough opponent to tell agent configurations apart (see Section 5.1), and any analysis of a real loss that was decided before the endgame.

`connectx/bitboard_solver.py` removes it. A position becomes two integers over a seven-bits-per-column layout, where one sentinel bit per column stops shifted patterns from wrapping between columns; win detection is four shift-and-mask operations instead of a scan over every cell and direction. The key is position + mask, which is a bijective encoding of a Connect-4 position, so the transposition table is keyed on one integer rather than a 42-element tuple. Crucially the table stores LOWER and UPPER BOUNDS rather than plain values, so a cutoff found under one alpha-beta window still prunes under a different one -- the older memo caches only values produced under whatever window happened to be live, which is why it must rebuild a fresh memo on every call. Moves that hand the opponent an immediate win are pruned before search, a position with two distinct forced replies is resolved with no recursion at all, and moves are ordered by how many new threats each creates rather than merely centre-out.

Measured against the older solver on the same positions: 50 to 70 times faster, and it resolves midgame positions with six or more open columns that the older one could not resolve at a 900-second budget.

HOW IT WAS VALIDATED is the part worth reporting. The first version agreed with the deployed solver on 37 of 40 randomly reached endgame positions -- a residual that reads like edge-case noise in fresh code. It was not. A third implementation was written specifically to adjudicate (`connectx/solver_referee.py`: plain minimax, NO alpha-beta at all, win detection via the environment cell scan, sharing no machinery with either solver so a bit-layout error could not be reproduced), and it judged all three disagreements: the NEW solver was wrong in 3 of 3, the deployed one right in 3 of 3. The cause was a single line -- the position update must assign the newly played stone to the player who just moved, and writing it the other way hands one's own stone to the opponent on every ply. After the fix, 40 of 40.

The general rule this earns: two solvers disagreeing tells you one is wrong but not which, and the deployed one being deployed is only evidence that nothing has caught it yet. An adjudicating third implementation that shares no machinery costs about eighty lines. A high agreement rate is not a pass -- the residual is where the bug lives.

2.4b Resisting in a lost endgame -- a measured consequence of value-only solving

The older solver returns only +1, 0 or -1. It carries no information about HOW FAST a result arrives. That is sound for deciding whether a position is won, and it is what the deployed agent has always used, but it has a consequence nobody had examined: when every move loses, every move scores identically, so the solver picks arbitrarily -- frequently choosing the fastest way to lose. Against perfect play that costs nothing, since the game is lost either way. Against the imperfect agents a real ladder is full of, it discards every chance the opponent errs before converting.

The bitboard solver's score encodes distance, so its argmax already prefers the slowest loss and the

fastest win. `_resisting_endgame_solve` wraps it in the older solver's exact signature. It was then measured rather than assumed: positions were sampled that the solver PROVES are losses, and each was played out twice from the identical position against the identical opponent with paired seeds -- once choosing our moves with the value-only solver, once with the distance-aware one. An escape is any non-loss, since from a proven-lost position a draw is equally the opponent failing to convert.

opponent	legacy escapes	resisting escapes	game length
perfect play (control)	0.000	0.000	+4.3 plies
weak heuristic	0.100	0.300	+3.5 plies
stronger heuristic	0.150	0.225	+3.8 plies

The control decides whether the rest means anything: against an opponent that never errs both arms escape exactly 0.000, because the positions really are lost -- while the resisting arm still survives 4.3 plies longer. The mechanism does what it claims without manufacturing false escapes. At $n=40$ the $+0.200$ against the weak heuristic is solid and the $+0.075$ against the stronger one is directional only. The option is opt-in and is NOT wired into the shipped `submission.py`: it is offered as a measured option, not a default.

2.5 Episodic memory, LoRA fine-tuning, and online learning

Three more general-purpose mechanisms were applied on top of the search:

- Episodic memory: a k-NN lookup over real self-play trajectories, both won and lost (a negative/repulsion signal, not just positive examples), blended into the value estimate at decision time [3][4]. Small but real positive effect vs. random play; no measurable effect vs. either fixed heuristic opponent.
- LoRA self-play fine-tuning: capacity-constrained low-rank deltas applied to the value head during self-play [5], rather than an unconstrained further gradient update -- 75.0% to 81.7% win rate vs. the stronger heuristic.
- Curriculum self-play: fixing a real gap where the stronger heuristic was never actually mixed into self-play training at all -- 81.7% to 85.0%, the best-confirmed result for this domain.
- Best-effort online learning (value head only, updated live as real games are played): isolated via a clean A/B test (the same submission loaded twice, one copy's update call disabled) -- currently measures exactly 0.0% effect against the synthetic test-opponent set at evaluation scale. Not evidence it is useless in general -- Kaggle's real opponent pool is far wider than three fixed synthetic opponents -- just an honest, now-measured answer.

3. Every bug found and fixed

Seven real, generic-class bugs were found and fixed during development. Each is recorded here because several are the kind of mistake that is easy to make again in a different adversarial domain.

#	Bug	Found via
1	Internal eval reported a false 100% win rate (done was assumed to imply a win)	Tracing one "solved" game through an independent alternating-turn harness
2	The value head never saw a losing example (unsolved walks were silently discarded)	Root-cause tracing after bug 1
3	Every prediction in the packaged submission was silently corrupted (a spurious extra ReLU)	Diffing the packaged output directly against the real model, node by node

4	"Same seed" training runs were not actually reproducible (unseeded global RNG)	Two same-seed runs produced different results
5	A whole session's A/B comparisons were confounded (one shared RNG across a batch)	An opening-move-only change swung the overall win rate by double digits
6	A left-column tie-break bias with no game-theoretic basis	Direct user-observed pattern in real play
7	The online learner taught the value head that losing and drawing are equally bad	Mining real Kaggle replay data via the Kaggle API for missed blocks

The generalizable lesson: bugs 2 and 7 are the same mistake, made twice, in two different places. "Treat a loss and a draw identically" is a natural default everywhere ELSE in a single-agent pipeline (every prior puzzle domain has exactly one failure mode -- unsolved -- with no distinction worth making), and it takes deliberate effort to remember that an adversarial domain has two qualitatively different bad outcomes.

4. Case study: diagnosing a real loss (zugzwang), and a first attempt at a fix

A live, observed loss -- an opponent slowly building what looked, in hindsight, like an obviously winning diagonal, with the agent apparently ignoring it -- was root-caused not by guesswork but by pulling the actual Kaggle replay and solving the real endgame exhaustively, working backward through the game.

Finding: the position was already a forced loss with 23 empty cells still on the board -- at least 15 real plies before the deployed 2-round search could possibly have seen it coming. The mechanism is a genuine, textbook Connect-4 ZUGZWANG / PARITY TRAP (classic odd/even threat theory [6]): once only two columns remained legal for an extended stretch, which player is forced to place the fatal piece is decided purely by the parity of total remaining cells across those two columns -- invisible to any bounded-depth positional search, and not addressed by the exact endgame solver above (its own calibrated safe zone starts well inside the window where this trap was already unavoidable).

4.1 A parity-heuristic attempt -- tried, measured, and honestly a mixed result

A column-parity feature was built and tested directly, rather than left purely theoretical: for each still-open column with r empty cells, under naive same-column-only alternation the player who would place the TOP piece is the mover if r is odd, the other player if r is even. This gives a cheap, real-valued "parity cost" per board position -- more even-parity open columns scored as worse for the agent -- blended into the leaf value estimate at a tunable weight.

This is explicitly NOT full Claimeven: a genuine claimeven strategy requires REACTIVE move-pairing enforced across an entire game, not a one-shot column count at a single position. It was built and measured as a testable nudge, not assumed to work because the theory motivates it.

Measured against the trusted test harness at several weights: the feature produces a REAL, repeatable effect, but not a clean win. At weight 0.5, win rate vs. the stronger heuristic rose from 70.0% to 77.5% -- but win rate vs. random-legal play fell from 100.0% to 90.0% in the same run. Smaller weights (0.1-0.3) showed the same pattern at reduced magnitude in both directions. Net honest verdict: this specific heuristic trades performance against weaker opponents for a partial, not fully consistent gain against a stronger one -- a real, measured signal in the direction the theory predicts, but not yet a confirmed fix. It ships in this repository as an opt-in, OFF-by-default parameter (`parity_weight=0.0` in `adversarial_search.py``) rather than a new default, exactly so a reader can

reproduce this exact finding rather than take it on faith.

Two angles remain genuinely untried: a value head deliberately trained on zugzwang-rich self-play positions (so the pattern becomes an implicit learned feature rather than a hand-tuned scalar nudge), and a full, correctly reactive Claimeven implementation rather than a single-position heuristic proxy.

5. Results

All win rates below are against an independent, trusted test harness (an alternating-turn engine that does NOT reuse the environment's own bundled step()) -- exactly the shortcut a submission validator needs to avoid). Three opponents throughout: random-legal play, the fixed weak heuristic the model trained against, and a 1-ply-deeper "stronger" heuristic never seen during training.

Configuration	vs random	vs weak heur.	vs stronger heur.
Latent beam search, depth=1 (original)	63.3%	0.0%	6.7%
Latent beam search, depth=3	55.0%	0.0%	1.7%
Real adversarial search (1 round)	96.7%	100.0%	66.7%
+ memory + online learner (bug 7 present)	98.3%	78.3%*	50.0%
+ bug 7 (loss-penalty) fixed	100.0%	100.0%	80.0%
+ LoRA self-play fine-tune	-	-	81.7%
+ curriculum self-play (best-ever)	-	-	85.0%
+ exact endgame solver (current, deployed)	100.0%	100.0%	83.3%

* The weak-heuristic dip in that one row is explained, not a mystery: that harness ran all opponent blocks sequentially in one process, so the online learner had already drifted from 60 preceding random-opponent games by the time it reached this block.

On Kaggle's own rating (a TrueSkill-style Gaussian score that starts uncertain and converges over dozens of real games against other real submissions): a freshly-uploaded submission's rating is not comparable to one that has had a day to settle. Confirmed directly during development -- a previously-deployed submission itself started near 300 right after upload and climbed past 450 only after 27 real games. 7x6 Connect-4 is a mathematically SOLVED game (the first player wins with perfect play) [6], so the real competitive pool likely includes near-perfect solvers -- the results above demonstrate the ARCHITECTURE works on this domain; they should not be read as a leaderboard-rating prediction.

5.1 The evaluation was saturated -- and what a discriminating opponent showed

The win rates above are measured against heuristic opponents, and by 2026-08-30 they had stopped carrying information: the single-member baseline scored 20/20, 20/20 and 19/20, and a held-out opponent suite returned 1.000 on five of six opponents including its strongest. A comparison between two agent configurations could produce at most a one-game difference. Every mechanism result on this domain was being measured against a ceiling.

With the bitboard solver available, a genuinely stronger opponent could be built: the same stronger heuristic used above, with PROVEN-OPTIMAL play layered on top wherever its budget allows. It reports the fraction of its own moves it actually proved, because an opponent that proved one move in seven is a hybrid and calling it perfect play would manufacture exactly the false ceiling it exists to remove. Uniformly random legal play scores 0.000 against it, confirming it is genuinely strong rather than merely different, and both agents come off the ceiling where the old suite had pinned them at 1.000.

One caution learned the hard way, and it is general: the opponent was initially budgeted in WALL-CLOCK time, so on a quieter machine it proved more moves and became a stronger opponent. One seed scored 0.812 at coverage 0.14 and 0.875 at coverage 0.20 under identical flags -- the measuring stick moved between measurements, producing first a false positive and then a false retraction. Switching to a node budget made cells reproducible exactly. Before trusting any evaluation, check whether any part of it is budgeted in wall-clock time.

6. Honest limitations

- The zugzwang-avoidance problem (Section 4) is diagnosed and a first fix attempt is measured, but not solved -- see Section 4.1's own honest verdict.
- Online learning's real-world value is unconfirmed -- it measures zero effect against three fixed synthetic opponents, which is not the same as zero effect against Kaggle's actual, much wider real pool.
- Deeper search beyond 2 rounds (without the endgame solver) was tried and found WORSE, not just unexplored -- unpruned 3-round search blows the time budget, and the only pruning width that stayed safe tested worse than plain 2-round search.
- The real Kaggle competitive pool likely includes near-perfect solvers, since 7x6 Connect-4 is a solved game -- see Section 5's closing note.

References

- [1] Schrittwieser, J. et al. "Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model." Nature, 2020.
- [2] Silver, D. et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play." Science, 2018.
- [3] Blundell, C. et al. "Model-Free Episodic Control." arXiv:1606.04460, 2016.
- [4] Pritzel, A. et al. "Neural Episodic Control." ICML, 2017.
- [5] Hu, E. J. et al. "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685, 2021.
- [6] Allis, L. V. "A Knowledge-Based Approach of Connect-Four." M.Sc. thesis, Vrije Universiteit Amsterdam, 1988. (First complete game-theoretic solution of Connect-4; the classical source for odd/even threat and zugzwang theory referenced in Section 4.)
- [7] Mnih, V. et al. "Human-level control through deep reinforcement learning." Nature, 2015. (Experience-replay precedent used by the on-policy Monte Carlo value-training loop.)
- [8] Adam, Addison Howard, and Bovard Doerschuk-Tiberi. "Connect X." <https://kaggle.com/competitions/connectx>, 2020. Kaggle.
- [9] Kaggle. "kaggle_environments." <https://github.com/Kaggle/kaggle-environments>. (Board/config schema and fixed-opponent-baked-into-step convention are ported from this package's own design, not from any published agent's code.)